

AJS Control — System Documentation

Technical reference · Version 1.0 · July 2026 · Internal & Confidential

1. System Overview

AJS Control is an internal operations platform for AJS and its three subsidiaries: Logistics, Bonded Warehouse, and Machinery. Its architectural core is a single generic monitoring spine — a **watched_items** table paired with an **alerts** table and a scheduled **alert engine** — into which every subsidiary module feeds. Anything with a deadline or threshold (permits, trips, invoices, extensions, machine idle time, bonded storage limits, driver licenses) becomes a watched item; the engine evaluates all of them every 5 minutes and raises prioritised alerts.

Design principle: *build the rule logic once, generically; subsidiary modules only feed data into the spine.* No module contains its own alert logic.

2. Technology Stack

Layer	Technology	Notes
Database	PostgreSQL 16 (Docker)	All timestamps stored UTC; port 54329 local, 54330 test
ORM / migrations	Prisma	12+ migrations, production-safe deploy via prisma migrate deploy
API	Node.js + TypeScript + Express	Port 4000; serves REST endpoints + alert engine cron
Frontend	Next.js (React) + Tailwind CSS	Port 3010; self-hosted fonts (no build-time network dependency)
Auth	JWT, 8h expiry, localStorage	bcrypt password hashing; login rate-limited 5/15min per IP
Scheduler	node-cron	Alert engine sweep every 5 min; disabled only if DISABLE_ALERT_CRON=true
Deployment	Docker Compose + Nginx	Production files built; TLS via certbot placeholder; not yet deployed

3. Core Data Model

3.1 The Spine

Table	Purpose	Key fields
watched_items	Everything under deadline/threshold watch	subsidiary, item_type, reference_id, status, due_at, threshold_value, owner_user_id
alerts	Raised alerts, one open max per watched item	watched_item_id, subsidiary, priority (low/medium/high), message, status (open/acknowledged/resolved)
alert_events	Append-only audit log (DB triggers block UPDATE/DELETE)	alert_id, event_type (created/acknowledged/resolved/reopened/auto_resolved), actor (user UUID or 'system'), note, created_at
alert_thresholds	Director-editable rule numbers	rule_key, value, updated_by, updated_at — read by engine each run

engine_heartbeats	Engine self-monitoring	run_at, status (success/failed), error_message, items_processed, alerts_created
notification_log	Every delivery attempt	alert_id, channel (email/sms), recipient_user_id, status, error_message

3.2 Subsidiary Tables

Module	Tables	Feeds spine as
Logistics	drivers, vehicles, trips, permits, maintenance_records	permit (incl. driver license expiry), trip, maintenance
Warehouse	movements (+ chassis_number, bond_value_tzs, vehicle_description), extensions, invoices	extension (incl. 180-day bonded storage deadline), invoice
Machinery	machines (status: waiting/successful/unsuccessful/idle)	machine_status (idle-days threshold)
Shared	users (role, subsidiary, phone, is_active, driver link)	—

Movement uniqueness is per **chassis_number**, not TANSAD reference_number (real customs data shares one TANSAD across multiple items). item_category values: car, truck, machine, bus.

4. Alert Rules (default thresholds — editable in Settings)

Rule	High	Medium	Auto-resolve
Permit / driver license expiry	within 48 hours	within 7 days	expiry extended
Trip overdue	in_progress past expected end	— (by design)	trip completed
Extension deadline	within 2 days	within 5 days	extension updated / exited
Invoice overdue	overdue > 7 days	overdue 1–7 days	paid_date set
Machine idle	idle > 5 days	idle > 2 days	status changes off idle
Bonded storage (180-day limit)	within 2 days of limit / over	within 5 days	exit_date set

All deadline arithmetic is computed in **Africa/Dar_es_Salaam (EAT, UTC+3)**. Date-only fields expire at 23:59:59 EAT. Storage in UTC, business logic and display in EAT — verified with boundary tests at 47h45m/48h15m and 180/181 completed EAT calendar days. Alerts auto-resolve with actor='system' when their condition clears; manual acknowledgment/resolution records the real user UUID. Duplicate prevention: no new alert if an open one exists for the same watched item.

5. Roles & Access Control

Role	Scope
Director / General Manager	All subsidiaries; user management; alert thresholds; all reports
Manager	Own subsidiary only (API-enforced 403 on others)
Accountant	Warehouse financial data; invoices/movements reports; no full alerts export
HR / Staff	Scoped per assignment
Driver	Own trips + assigned vehicle + license reminder only, via separate portal; 403 on all else

Deactivation is soft (is_active=false): login blocked, notifications stopped (recipient query filters is_active), audit history preserved. Hard deletes are not used anywhere user references exist.

6. Operations Runbook

6.1 Local development

docker compose up -d (Postgres, port 54329) · npm run api:build / npm run build · frontend http://127.0.0.1:3010 · API http://127.0.0.1:4000. Secrets live in .env (gitignored): DATABASE_URL, JWT_SECRET, SMTP_*, AT_* (Africa's Talking), WEB_ORIGIN.

6.2 Testing

npm run db:test:up → db:test:migrate → test:db. Regression suite covers all 8 rule types, duplicate prevention, auto-resolve, role scoping. It runs ONLY against the isolated test DB (port 54330, .env.test) — it seeds residue and must never run against operational data. There is deliberately no ambiguous 'npm test'.

6.3 Health monitoring

GET /api/health/engine (unauthenticated) returns latest heartbeat; stale=true if no success within 2× the run interval (10 min). Settings shows the same status for directors. Recommended: point an external uptime monitor at this endpoint once deployed.

6.4 Data import

CSV importers (dry-run first, then commit): drivers/vehicles via import:logistics; bond stock movements via import-bond-stock (unique by chassis). All importers validate enums/dates and neutralize CSV formula injection on export (cells starting with = + - @ tab/CR are quoted). Category/status values are strict allow-lists.

6.5 Production deployment (prepared, not yet executed)

Dockerfile + docker-compose.prod.yml (Postgres volume, API+frontend internal, ports 80/443 only, TZ=UTC pinned, health checks) · Nginx reverse proxy with security headers, certbot-ready · scripts/deploy.sh (git pull, prisma migrate deploy, rebuild, restart) · scripts/backup.sh (pg_dump, timestamped, for nightly cron). On first server run: sudo nginx -t and docker compose -f docker-compose.prod.yml config.

7. Security Posture

Area	Status
Passwords	bcrypt hashed; strict login rate limit (5/15min/IP, verified 429)
Tokens	JWT 8h, localStorage — reviewed; acceptable for small internal tool; no XSS sinks found (no dangerouslySetInnerHTML/innerHTML/eval; React escaping throughout)
Audit integrity	alert_events append-only, enforced by DB trigger (UPDATE/DELETE blocked)
CSV exports	Formula-injection neutralized centrally in createCsv
Secrets	In gitignored .env; verified absent from git history
Network	All services bind 127.0.0.1 locally; production exposes only 80/443 via Nginx
Error handling	Malformed requests return clean 4xx JSON, not stack traces

8. Current State & Known Gaps

Item	State
Warehouse data	LIVE — 158 real bond stock movements (as at 30 Jun 2026); 18 over-limit + 5 warning items correctly alerted (TZS 727M bond value over limit)
Logistics data	Fixtures only — real driver/vehicle registers pending collection
Machinery data	Fixtures only
SMS (Africa's Talking)	Code complete & verified fail-safe; awaiting credentials wiring
Email (SMTP)	Code complete; Gmail app-password setup pending
Server deployment	Files ready; VPS not yet provisioned (Oracle Free Tier recommended)
GPS (Cartrack)	Test script ready; account awaits Fleet API enablement (fleet-api@cartrack.com)
Record detail pages	Modals exist for all 9 record types; dedicated full pages deliberately deferred

9. Maintenance Principles

- New alert type? Add a rule function + threshold rows + feed watched_items from the module — never bespoke alert logic in a module.
- Run npm run test:db after every change.
- Alert thresholds are business configuration (Settings), not code.
- Never hard-delete users or alert events.
- The heartbeat endpoint is the first thing to check when 'no alerts' seems suspicious.
- Dry-run every import before committing.
- This document should be updated when the schema or rules change — it is part of the system.